

Отчёт по лабораторной работе №21

по курсу «Алгоритмы и структуры данных».

Выполнил студент группы М8О-114БВ-25 Степанова Анастасия Михайловна № по списку 21.

Контакты: Nastya.stepanova1104@yandex.ru

Работа выполнена: «05» марта 2026 г.

Преподаватель: каф.806 Сысоев Максим Алексеевич

Входной контроль знаний с оценкой: _____

Отчет сдан «10» марта 2026 г.

Итоговая оценка: _____

Подпись преподавателя: _____

1. **Тема:** Программирование на интерпретируемых командных языках ОС UNIX.

2. **Цель работы:** Разработать сценарий (скрипт) для командного интерпретатора Bash и программу на языке Python для удаления всех синонимов (жёстких ссылок) указанного файла в заданном каталоге и всех его поддиректориях.

3. **Задание (Вариант № 21):** Удаление всех синонимов указанного файла из указанного каталога и его поддиректорий.

4. **Оборудование:** Оборудование ПЭВМ студента, если использовалось:

Процессор: Intel(R) Pentium(R) CPU 6405U @ 2.40GHz 2.40 GHz, ОП 8 ГБ, SSD 256 ГБ

5. **Программное обеспечение:** Программное обеспечение ЭВМ студента, если использовалось:

Операционная система семейства Unix, наименование Ubuntu, версия 24.04 LTS, интерпретатор команд `bash` версия `5.2.21`.

Система программирования: gcc (Ubuntu 11.4.0-1ubuntu1~22.04.2) версия 11.4 Редактор текстов: VS code версия: 1.107.1.

Язык программирования: Python, Bash

6. **Идея, метод, алгоритм решения задачи [в формах: словесной, псевдокода, графической (блок-схема, диаграмма, рисунок, таблица) или формальные спецификации с пред- и постусловиями]:**

Идея: В файловых системах UNIX каждая жёсткая ссылка (синоним) указывает на одну и ту же inode. Следовательно, идентифицировать файлы как синонимы можно по совпадению номера inode. Задача сводится к сравнению номеров inode всех найденных файлов с номером inode целевого файла.

Метод:

1. **Получение аргументов:** Скрипт принимает два параметра: путь к исходному файлу и путь к каталогу для поиска.
2. **Определение inode оригинала:** Получаем абсолютный путь к исходному файлу (для избежания ошибок с относительными путями) и извлекаем его номер inode.
3. **Рекурсивный обход:** Используя стандартные средства (команда `find` в Bash и функция `os.walk()` в Python), выполняем рекурсивный обход всех файлов и директорий внутри указанного каталога.
4. **Сравнение и удаление:**
 - Для каждого найденного файла получаем его номер inode.
 - Если номер inode найденного файла совпадает с номером inode исходного файла, это означает, что они являются жёсткими ссылками на одни и те же данные.
 - Перед удалением убеждаемся, что путь к найденному файлу не совпадает с путём к исходному файлу (чтобы не удалить оригинал).
 - Если это другой синоним, выводим сообщение о его удалении и выполняем команду `rm` (или `os.remove()` в Python).

Алгоритм:

1. `file` = аргумент1

2. dir = аргумент2
3. abs_file = абсолютный_путь(file)
4. inode_original = номер_inode(abs_file)
5. Для **каждого** found_path в рекурсивный_обход(dir):
6. Если found_path является файлом:
7. inode_current = номер_inode(found_path)
8. Если inode_current == inode_original:
9. Если абсолютный_путь(found_path) != abs_file:
10. напечатать("Удаление:", found_path)
11. удалить_файл(found_path)

7. Сценарий выполнения работы (план работы, первоначальный текст программы в черновике (можно на отдельном листе) и тесты либо соображения по тестированию).

План работы:

1. Анализ предметной области: изучить понятие жёстких ссылок (синонимов) в ОС UNIX.
2. Проектирование алгоритма: определить способ идентификации синонимов через inode.
3. Реализация на Bash:
 - Использовать realpath для нормализации путей.
 - Использовать ls -i и awk для получения номера inode.
 - Использовать find для рекурсивного обхода.
4. Реализация на Python:
 - Использовать модули os и sys.
 - Применить os.walk() для обхода.
 - Использовать os.stat().st_ino для получения номера inode.
5. Тестирование: проверка на различных структурах каталогов и файлах.

Тесты:

```
mkdir test_links
cd test_links
```

```
echo hello > file.txt
```

```
ln file.txt copy1.txt
ln file.txt copy2.txt
```

```
mkdir subdir
ln file.txt subdir/copy3.txt
```

```
ls -i
ls -i subdir
```

```
python3 script.py file.txt . // bash script.sh file.txt .
```

```
ls
ls subdir
ls -l
```

Соображения по тестированию:

- **Проверка на пустом каталоге:** Скрипт не должен ничего удалять и завершаться без ошибок.
- **Проверка с файлом, не имеющим синонимов:** Скрипт не должен ничего удалять.
- **Проверка с файлами в поддиректориях:** Убедиться, что обход работает рекурсивно.

Пункты 1-7 отчета составляются строго до начала лабораторной работы.

Допущен к выполнению работы. Подпись преподавателя _____

8. Распечатка протокола (подклеить листинг окончательного варианта программы с тестовыми примерами, подписанный преподавателем).

```
#!/bin/bash
```

```
help() {
echo "Usage: $0 [OPTION]... FILE DIR"
echo
echo "Delete all hard links (synonyms) of FILE inside DIR and its subdirectories."
echo
echo "Options:"
echo " -h, --help    show this help message and exit"
```

```

echo "Exit status:"
echo " 0  if OK,"
echo " 1  if invalid arguments or missing parameters"
echo " 2  if FILE does not exist"
echo " 3  if DIR does not exist or is not a directory"
}

if [[ "$1" == "-h" || "$1" == "--help" ]]; then
    help
    exit 0
fi
if [ $# -ne 2 ]; then
    help
    exit 1
fi
file=$1
dir=$2
if [ ! -f "$file" ]; then
    echo "Error: file '$file' does not exist"
    exit 2
fi
if [ ! -d "$dir" ]; then
    echo "Error: directory '$dir' does not exist"
    exit 3
fi
file=$(realpath "$file")
inode=$(ls -li "$file" | awk '{print $1}')
for f in $(find "$dir" -type f)
do
    inode2=$(ls -li "$f" | awk '{print $1}')
    file2=$(realpath "$f")
    if [ "$inode2" = "$inode" ] && [ "$file2" != "$file" ]; then
        echo "deleting $f"
        rm "$f"
    fi
done
///

#!/usr/bin/env python3

import os
import sys
import argparse

parser = argparse.ArgumentParser(
    description="Delete all hard links (synonyms) of FILE inside DIR and its subdirectories."
)
parser.add_argument("file", help="source file")
parser.add_argument("directory", help="directory to search")
args = parser.parse_args()
file = args.file
directory = args.directory
if not os.path.isfile(file):
    print(f"Error: file '{file}' does not exist")
    sys.exit(2)
if not os.path.isdir(directory):
    print(f"Error: directory '{directory}' does not exist")
    sys.exit(3)

```

```

file = os.path.abspath(file)
inode = os.stat(file).st_ino
for root, dirs, files in os.walk(directory):
    for name in files:
        path = os.path.join(root, name)
        file2 = os.path.abspath(path)
        inode2 = os.stat(path).st_ino
        if inode2 == inode and file2 != file:
            print("deleting", path)
            os.remove(path)

```

9. **Дневник отладки** должен содержать дату и время сеансов отладки и основные события (ошибки в сценарии и программе, нестандартные ситуации) и краткие комментарии к ним. В дневнике отладки приводятся сведения об использовании других ЭВМ, существенном участии преподавателя и других лиц в написании и отладке программы.

№	Лаб. или дом.	Дата	Время	Событие	Действие по исправлению	Примечание

10. **Замечания автора** по существу работы: замечания отсутствуют.

11. **Выводы:** В ходе выполнения лабораторной работы были разработаны два скрипта (на Bash и Python) для удаления синонимов (жёстких ссылок) заданного файла в указанном каталоге и его поддиректориях. Алгоритм работы основан на сравнении номеров inode файлов. Реализована защита от случайного удаления исходного файла. Оба скрипта успешно протестированы в среде ОС Ubuntu. Работа позволила закрепить навыки рекурсивного обхода файловой системы, работы с системными вызовами для получения атрибутов файлов и обработки исключительных ситуаций при программировании на интерпретируемых языках.

Подпись студента _____